Orion V
CIS 25
11/2/2026

CIS25 Midterm Project: Hexapod Servo Motor Calibration Tool

1. Program Description

What it does:

This program is a calibration tool for a hexapod robot. The robot has six legs, each with three servo motors (coxa/hip, femur/upper leg, tibia/lower leg) and a microswitch on the foot, for 18 servos and 6 foot switches in total.

A servo's position is set by the width of an electrical pulse, measured in microseconds (μs). In theory, a 1500μs pulse puts every servo at its center. In practice, each servo is mounted slightly off, because the servo horn only fits on the shaft in fixed steps, and small differences in manufacturing and 3D printing add up. To correct this, each servo gets a trim value. Trim is the small offset added to every pulse so the joint lands where the code expects. If the trims are wrong, the robot leans and its feet don't touch down evenly.

The program simulates the hardware, loading every servo's settings and every foot switch's state from a text file, servo_config.txt. From the menu, the user can:

1. View every part of the robot
2. View the parts grouped by leg
3. Search for a servo by its ID number
4. Adjust a servo's trim, which is checked to ensure that the input value is safe
5. Toggle a foot switch to simulate a foot touching down or lifting off
6. See which feet are on the ground and whether the robot has enough contact to stand
7. View the size in bytes of each data type and class used
8. Save the updated calibration data back to the file

How it works:

The program uses five classes:

- **Component** is an abstract base class for every physical part. It holds an ID and a name, and requires every derived class to define a display() function.
- **Servo** (derived from Component) stores one motor's board channel, pulse limits, center pulse, and a trim. Its setTrim() function rejects any trim larger than 200μs or any trim that would push the pulse outside the servo's range.

- **Footswitch** (derived from Component) stores one foot's sensor's pin number and whether it's pressed.
- **Leg** groups three servos and one foot switch, using pointers. It doesn't own them, it only refers to parts stored in Hexapod class.
- **Hexapod** represents the whole robot. It owns all the parts, reads and writes the config file, and is the only class main.cpp talks to.

When the program starts, Hexapod::loadConfig() reads the file twice. The first pass counts the lines for each part and allocates memory. It then creates the servo and switch arrays with new[]. The second pass reads each line's values into those arrays. Both arrays are then sorted by ID with insertion sort, so they can be searched with binary search. Each leg is connected to its parts by searching for them by ID; leg i uses servos 3i, 3i+1, 3i+2 and foot switch i.

All 24 parts are also stored in one array of Component * pointers. Polymorphism is utilized when "Show all parts" loops through the array and calls display(), with each part automatically using its own version, servo or footswitch.

When the user searches for a servo, the binary search returns a pointer to the actual servo in the array, so changing its trim updates the real object, and the leg that points to it sees the change, too. When the program exits, the Hexapod destructor frees all the dynamically allocated memory.

2. Concepts Used

Concept: **Data types and sizes**

- I chose fixed-size integer types to match the hardware: uint8_t (1 byte) for IDs, channels and pins; uint16_t (2 bytes) for pulse widths; int16_t (2 bytes, signed) for trim, which can be negative. Menu option 7 uses sizeof to print the size of each type and class, plus the total dynamic memory in use.

Concept: **File types**

- The servo_config.txt file is a plain-text data file. Each line starts with S (servo) or F (footswitch), and lines starting with # are comments.

Concept: **File I/O**

- Hexapod::loadConfig() reads the file with ifstream, using getline and >>. Hexapod::saveConfig() writes the updated calibration back with ofstream in the same format, so saved changes load the next time the program runs.

Concept: **Pointers**

- Leg holds Servo * and Footswitch * pointers to parts owned by Hexapod. findServo() and findSwitch() return pointers so the caller can modify the real object. The this-> pointer is used in constructors. Component ** is used for the array of base-class pointers.

Concept: **Dynamic Memory**

- The servo array, foot switch array, and parts pointer array are created with new[], sized from the file at run time, and freed with delete[] in the destructor. Copying a Hexapod is disabled (= delete) so two objects can never free the same memory.

Concept: **Arrays**

- Dynamic arrays of Servo and Footswitch; a fixed array Leg legs[6], since a hexapod always has 6 legs; an array of Component * pointers; and a string array of leg names.

Concept: **Binary Search**

- Hexapod::findServo() and Hexapod::findSwitch() search the sorted arrays by ID. This is used in menu options 3, 4, and 5 and connects each leg to its parts during loading.

Concept: **Sorting**

- Insertion sort puts both arrays in ID order after loading, so binary search works even if the file is out of order.

Concept: **Strings**

- Part and leg names, the S/F/# line types read from the file, menu prompts, and output messages.

Concept: **Classes (5)**

- Component, Servo, Footswitch, Leg and Hexapod, each split into its own .h header and .cpp implementation file. They use private and protected members, getters and setters, constructors and destructors.

Concept: **Inheritance and polymorphism**

- Servo and Footswitch inherit from the Component class and override its pure virtual display() function. Menu option 1 displays all parts through Component * pointers.

Concept: **Input Validation**

- readInt() in main.cpp re-prompts if the user types something that isn't a number, and setTrim() rejects trim values outside the safe range.

3. Screenshots

Figure 1: Program start: the config file loads and the main menu appears.



Figure 2: Option 1, Show all parts: all 18 servos and 6 foot switches displayed through the Component * array.

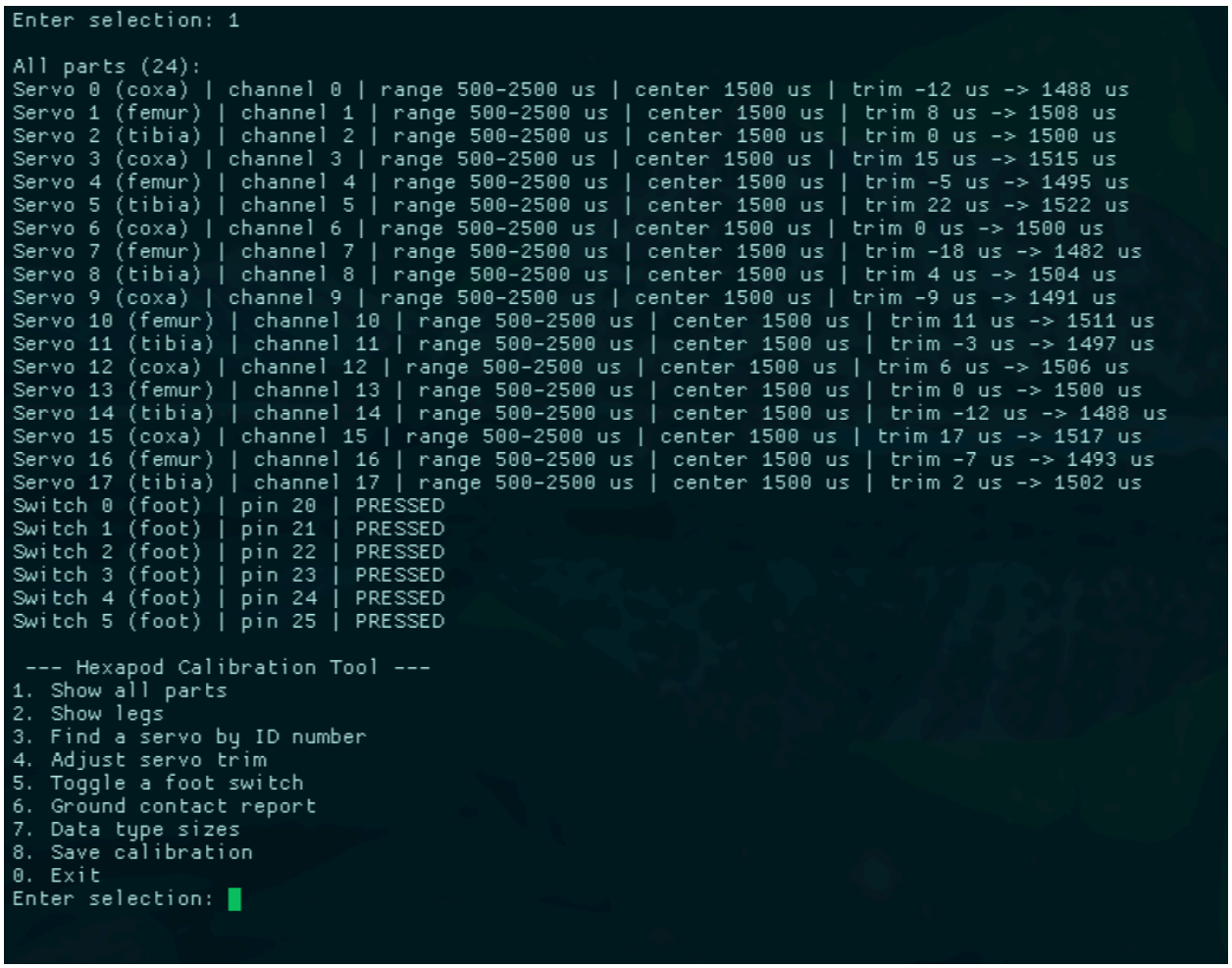


Figure 3: Option 2, Show legs: each leg with its three servos and foot switch.

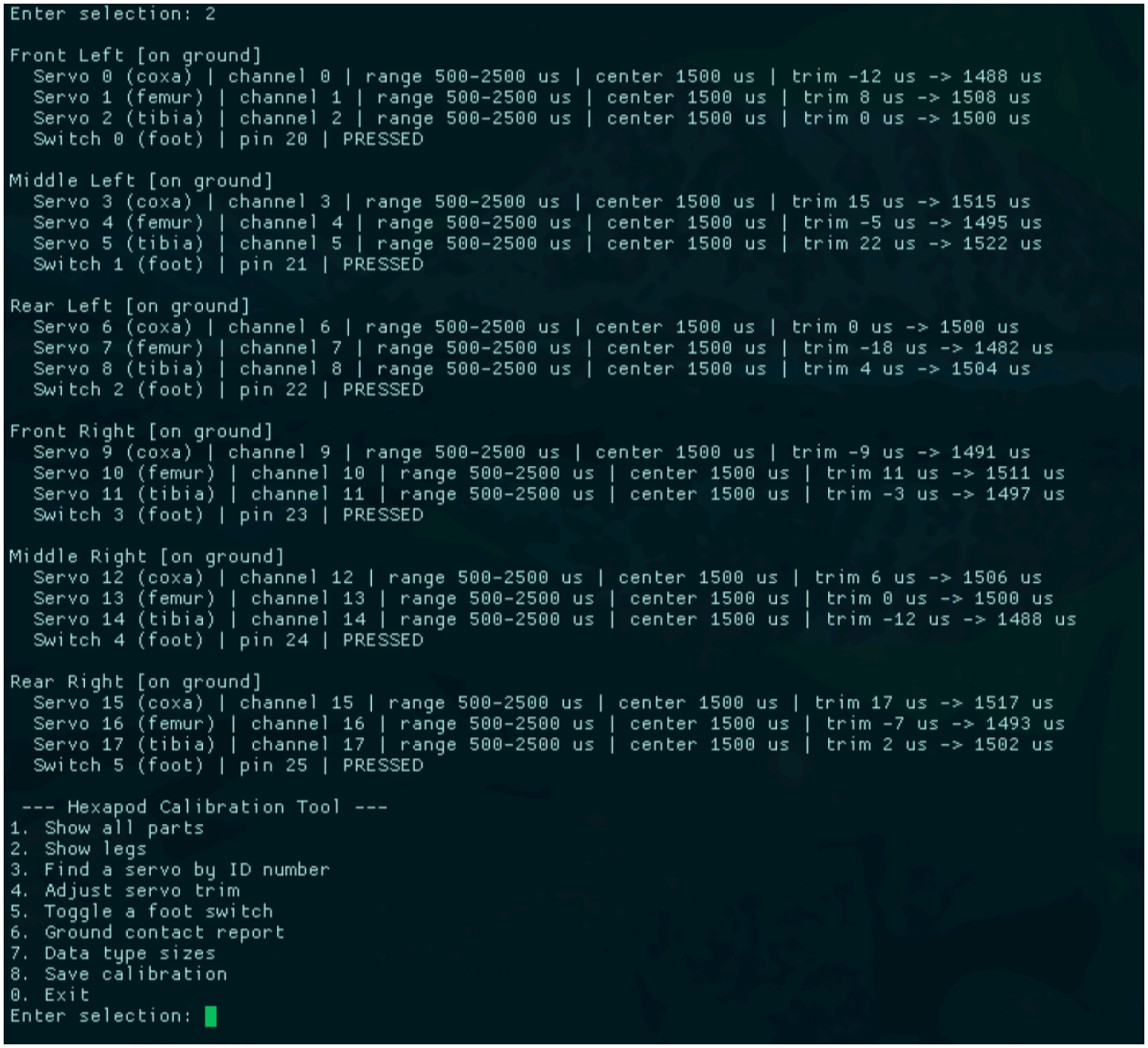


Figure 4: Option 3, Find a servo by ID: a successful binary search (ID 14).

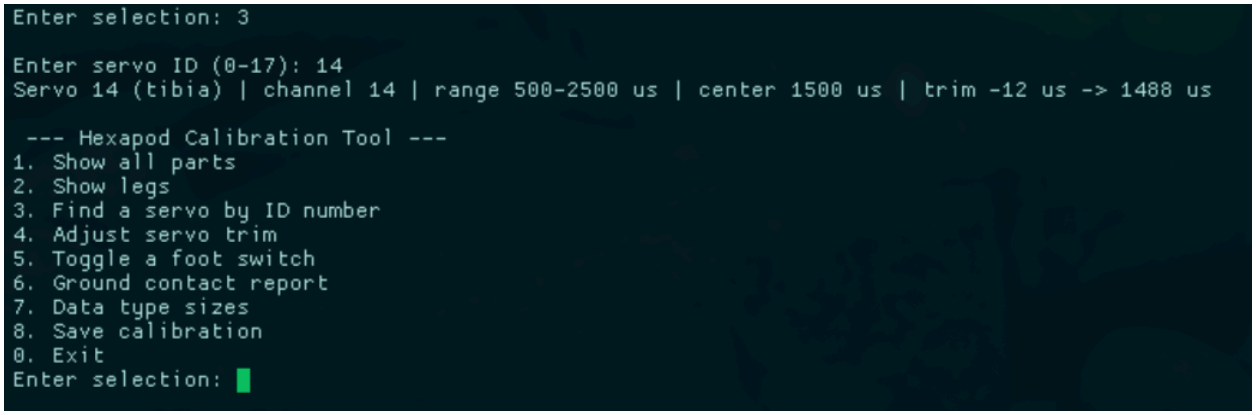


Figure 5: Option 3, Find a servo by ID: an ID that doesn't exist (ID 40) returns "not found".



Figure 6: Option 4, Adjust servo trim: a valid trim (−30) is accepted, and the new pulse is shown.

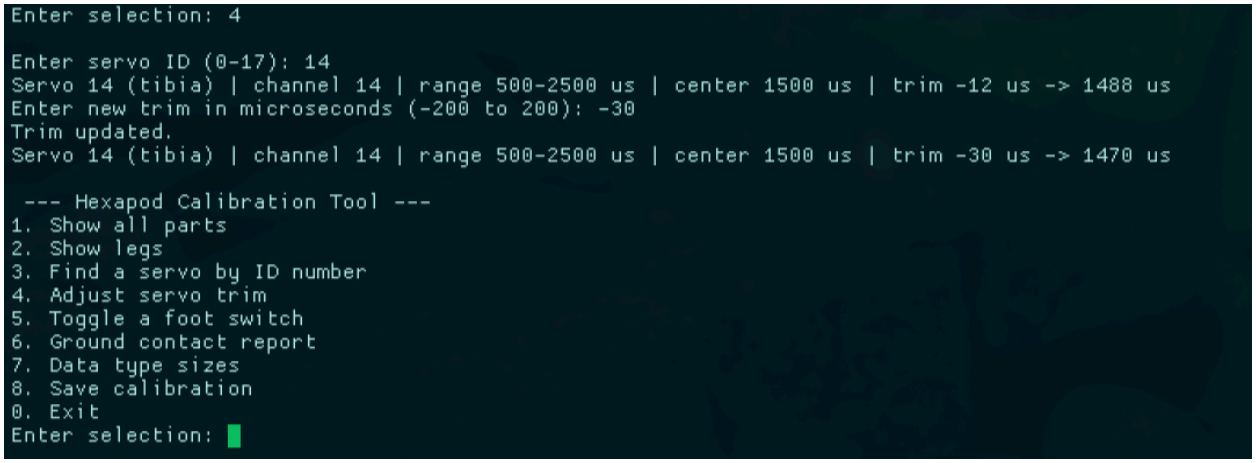


Figure 7: Option 4, Adjust servo trim: an unsafe trim (500) is rejected.



Figure 8: Option 5, Toggle a foot switch: switch 2 changes from PRESSED to released.

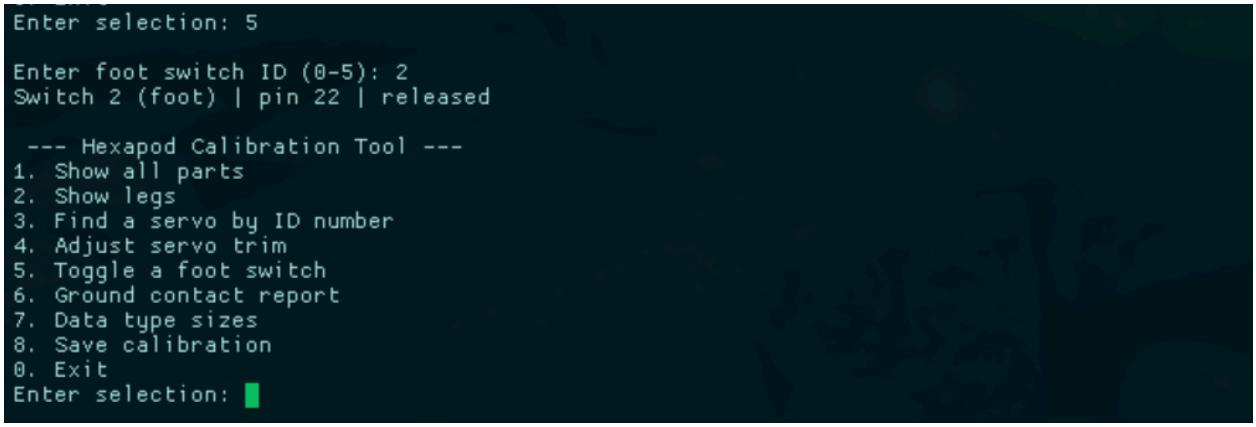


Figure 9: Option 6, Ground contact report: after three switches are toggled, the report shows 3 of 6 feet down and that the robot can stand.



Figure 10: Option 7, Data type sizes: the sizeof report for types, classes and dynamic memory.

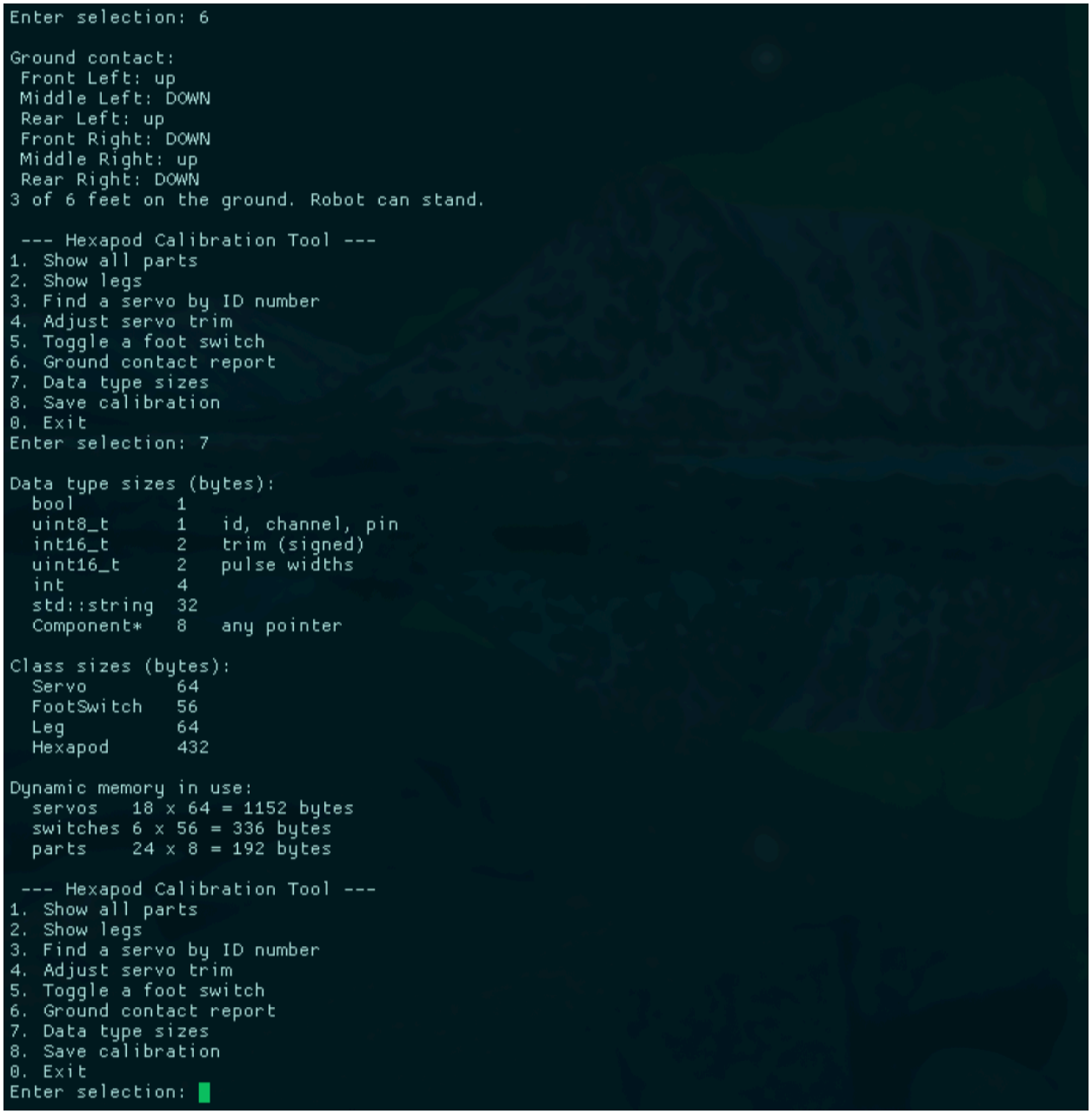


Figure 11: Option 8, Save calibration, followed by the updated line for servo 14 in servo_config.txt.

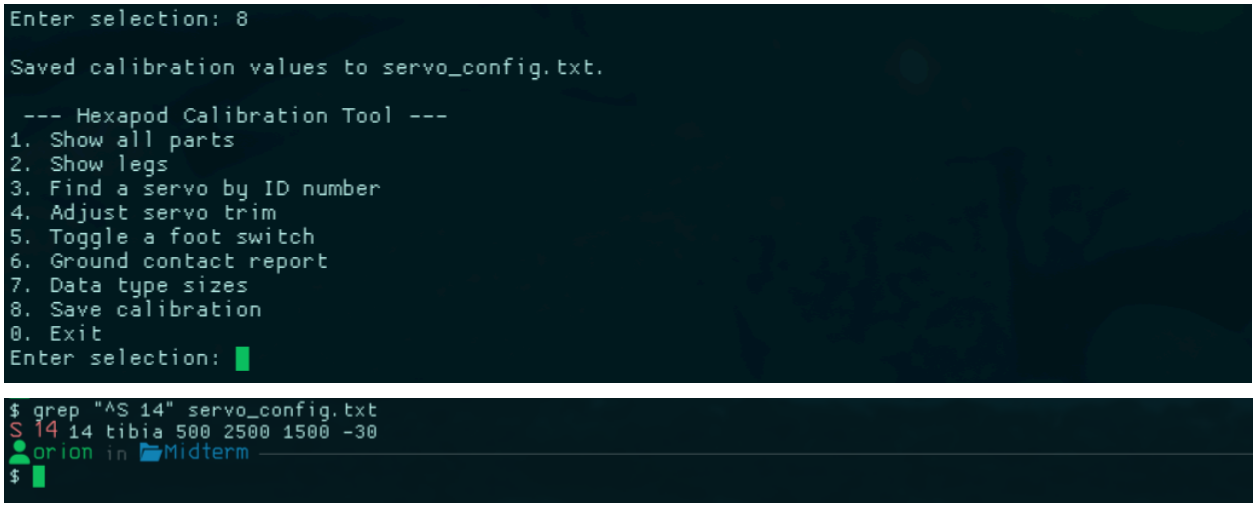


Figure 12: Restarting the program and finding servo 14 again: the saved trim loaded from the file.

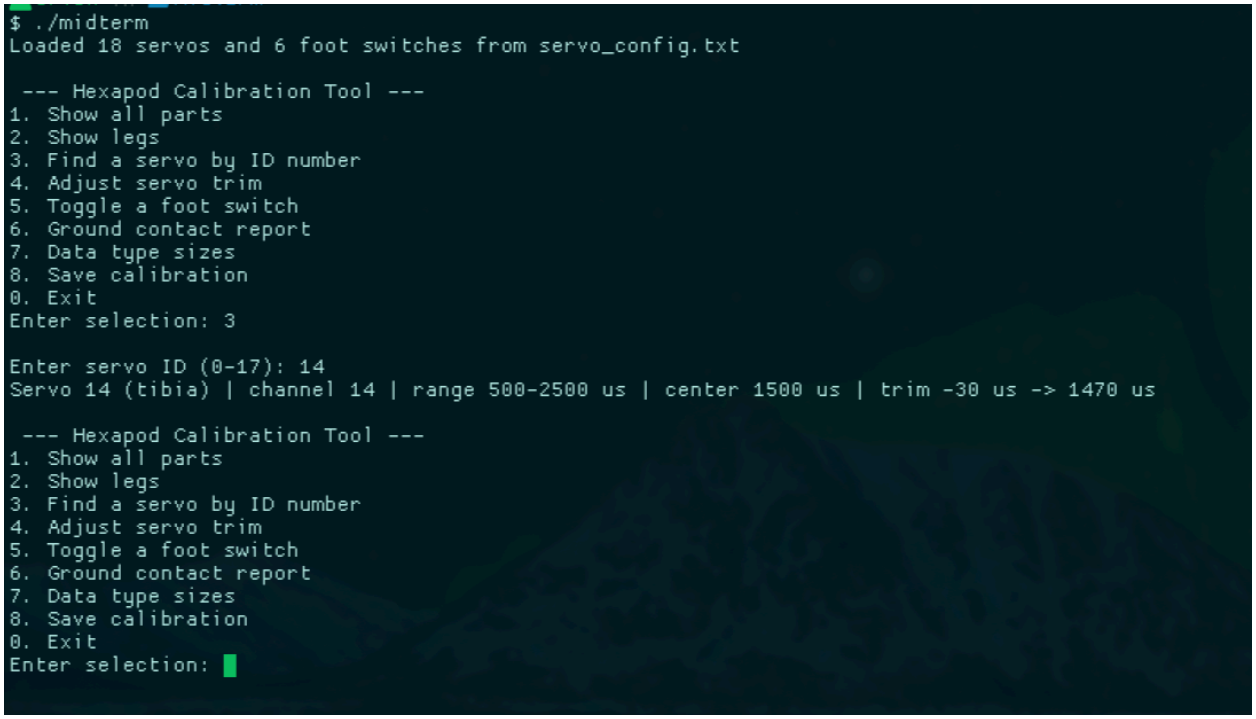


Figure 13: Input validation: typing letters at the menu re-prompts for a number.



Figure 14: Option 0, Exit.

